实验一 找最大和最小元素与归并分类算法实现（用分治法）

一、实验目的

1、掌握能用分治法求解的问题应满足的条件。
2、加深对分治法算法设计方法的理解与应用。
3、锻炼学生对程序跟踪调试能力。
4、通过本次实验的练习培养学生应用所学知识解决实际问题的能力。

二、实验内容

1、输入 n 个数，用分治法找出其中的最大和最小值。
2、输入 n 个数，用归并分类算法进行排序。

三、实验要求

1、能够根据分治法的思想，设计并实现求解数组中最大值和最小值的程序。要求程序能够处理不同规模的输入用例，准确输出最大值和最小值，并给出递归调用次数。

2、能够编写程序实现归并排序算法，并在程序中加入比较次数的计数功能，输出排序结果和比较次数。

四、实验过程设计（算法设计过程）

1、找最大和最小元素

分治法(Divide and couquer)将数组分成左右两部分，先求出左半部份的最大值和最小值，再求出右半部份的最大值和最小值，然后综合起来求

总体的最大值及最小值。这是个递归过程，对于划分后的左右两部分，同样复这个过程，直到划分区间内只剩一个元素或者两个元素。

问题可以简化为在 n 个数里面寻找最大和最小值。

（1）将数据等分为两组(两组数据的个数可能相差 1)，目的是分别选取其中的最大（小）值。

（2）递归分解直到每组元素的个数<=2，则可以简单地找到其中的最大（小）值。

（3）回溯时合并子问题的解，在两个子问题的解中大者取大，小者取小，即合并为当前问题的解。

2、归并分类

归并排序的实现过程可以分为两个主要部分：分解和合并。首先，将待排序的序列递归地分成若干个子序列，每个子序列包含一个元素；然后，逐步合并这些子序列，每次合并两个有序的子序列，直到合并为一个有序的序列。

算法流程：

（1）分解：将序列递归分成两半，直到每个子序列只包含一个元素。

（2）合并：将两个已排序的子序列合并为一个有序的序列。比较两个子序列的头部元素，选择较小的元素放入结果序列中，并移动指针到下一个位置。重复此过程，直到所有元素都被合并到一个序列中。

五、源代码

1、找最大和最小元素

```
// g++ 1_1.cpp -o 1_1 && ./1_1
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;
```

```
pair<ll, ll> minmax_divide(const vector<ll>& a, int l, int r, ll&
calls) {
    calls++; // 统计递归调用次数
    if (l == r) return {a[l], a[l]};
    if (r - l == 1) {
        if (a[l] < a[r])    return {a[l], a[r]};
        else    return {a[r], a[l]};
    }
    int mid = l + (r - l) / 2;
    auto L = minmax_divide(a, l, mid, calls);
    auto R = minmax_divide(a, mid + 1, r, calls);
    return {min(L.first, R.first), max(L.second, R.second)};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<ll> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    ll calls = 0;
    auto ans = minmax_divide(a, 0, n - 1, calls);
    cout << "最小值 = " << ans.first << "\n";
    cout << "最大值 = " << ans.second << "\n";
```

```
    cout << "递归调用次数 = " << calls << "\n";
    return 0;
}
```

2、归并分类

```
// g++ 1_2.cpp -o 1_2 && ./1_2
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;

void merge_sort(vector<ll>& a, vector<ll>& buf, int l, int r, ll& cmp) {
    if (l >= r) return;
    int m = (l + r) / 2;
    merge_sort(a, buf, l, m, cmp);
    merge_sort(a, buf, m + 1, r, cmp);
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        cmp++; // 仅统计元素间的大小比较
        if (a[i] <= a[j]) buf[k++] = a[i++];
        else buf[k++] = a[j++];
    }
    while (i <= m) buf[k++] = a[i++];
    while (j <= r) buf[k++] = a[j++];
    for (int t = l; t <= r; ++t) a[t] = buf[t];
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<ll> a(n), buf(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    ll cmp = 0;
    merge_sort(a, buf, 0, n ? n - 1 : 0, cmp);
    cout << "排序结果：";
    for (int i = 0; i < n; ++i) cout << a[i] << (i + 1 == n ? '\n' : ' ');
    cout << "比较次数 = " << cmp << "\n";
    return 0;
}
```

六、运行结果和算法复杂度分析

1、 找最大和最小元素

（1）运行结果



（2）复杂度分析：

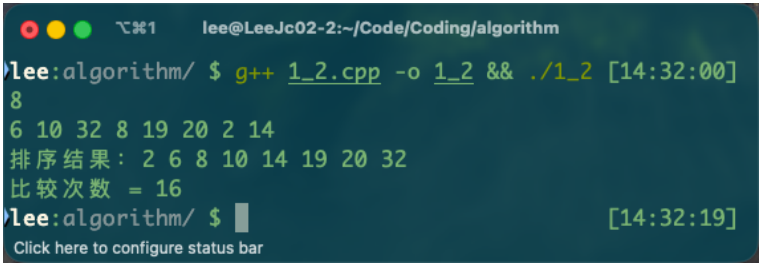
递推：T(n)=2T(n/2)+O(1)，T(1)=O(1)，得 T(n)=O(n)

时间复杂度：O(n)

空间复杂度：O(log n)

2、归并分类

（1）运行结果：



（2）复杂度分析：

递推：T(n)=2T(n/2)+n，T(1)=O(1)，得 T(n)=O(n log n)

时间复杂度：O(n log n)，**空间复杂度**：O(n)

实验二 背包问题和最小生成树算法实现（用贪心法）

一、实验目的

1、掌握能用贪心法求解的问题应满足的条件；
2、加深对贪心法算法设计方法的理解与应用；
3、锻炼学生对程序跟踪调试能力；
4、通过本次实验的练习培养学生应用所学知识解决实际问题的能力。

二、实验内容

1、使用贪心法实现部分背包问题算法。
2、使用贪心法实现最小生成树算法（Kruskal 和 Prim 至少选择一个）。

三、实验要求

（1）已知背包容量，物品数量、每个物品价值和重量，物品可以分割成任意大小。设计算法实现尽可能让装入背包中的物品总价值最大，但不能超过总容量。

（2）编写完整的 Kruskal 或 Prim 算法程序，能够读取图的输入数据（如邻接矩阵或邻接表），并输出最小生成树的边集及其总权值。

四、实验过程设计（算法设计过程）

贪心算法在对问题求解时，总是做出在当前看来是最好的选择。也就是说，不从整体最优上加以考虑，做出的是在某种意义上的局部最优解。

贪心算法不是对所有问题都能得到整体最优解，关键是贪心策略的选择，选择的贪心策略必须具备无后效性，即某个状态以前的过程不会影响以后的状态，只与当前状态有关。

1、背包问题

一般背包问题的目标是最大化装入背包的物品总价值，同时装入物品的总重量不能超过背包的容量。

选择的贪心策略为，每次选择单位重量价值最高的物品装入背包，直到达到背包容量或无法再装入更多物品为止。

算法执行过程如下所述。

（1）计算出每种物品的单位重量价值，即物品的价值除以重量。

（2）将物品按单位重量价值从高到低排序。

（3）依次选择单位重量价值最高的物品装入背包，直到达到背包容量或无法再装入更多物品。

2、最小生成树

使用贪心法求解最小生成树问题的实验过程主要包括 Kruskal 算法和 Prim 算法两种方法。这两种算法都是通过贪心策略来寻找图的最小生成树，确保生成树的总耗费最小。

（1）Kruskal 算法

Kruskal 算法的基本思想是对图中的所有边进行排序，然后从小到大选择边，确保不形成环。

具体步骤如下：

1）排序：将图中的所有边按照权重从小到大排序。

2）选择边：选择排序后的第一条边，检查其是否与已选择的边形成环。

3）重复步骤 2），直到选择了 V-1 条边（V 是顶点数）。

（2）Prim 算法

Prim 算法的基本思想是从图中的一个顶点开始，逐步增加边和顶点，直到包含图中所有顶点为止。

具体步骤如下：

1）初始化：选择一个顶点作为起始点，将其加入最小生成树集合 S，其余顶点属于 V-S。

2）寻找最短边：在 V-S 中找到与 S 中顶点距离最近的顶点 j，并将边(j, closest[j])加入最小生成树。

3）重复步骤 2），直到所有顶点都被加入 S。

五、源代码

1、背包问题

```
// g++ 2_1.cpp -o 2_1 && ./2_1
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;
struct Item {
    int id;
    int weight;  // 注意：改为先存重量
    int value;   // 再存价值
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n; int W;
    cin >> n >> W;

    vector<Item> a(n);
    for (int i = 0; i < n; ++i) {
        int w, v;
        cin >> w >> v;
        if (w < 0 || v < 0) {
```

```
            cout << "输入不合法\n";
            return 0;
        }
        a[i] = { i, w, v };
    }
    // 0/1 背包：一维 dp + 二维选择表用于回溯方案
    vector<ll> dp(W + 1, 0);
    vector<vector<char>> take(n + 1, vector<char>(W + 1, 0));

    for (int i = 1; i <= n; ++i) {
        int w = a[i - 1].weight;
        int v = a[i - 1].value;
        for (int c = W; c >= w; --c) {
            ll notake = dp[c];
            ll takeit = dp[c - w] + v;
            if (takeit > notake) {
                dp[c] = takeit;
                take[i][c] = 1; // 在(i,c)选择了第 i 件
            }
        }
    }
    // 回溯出一组选择
    int c = W;
    struct Pick { int id; double takeWeight, takeValue, fraction; };
    cout << "最大总价值 = " << static_cast<ll>(dp[W]) << "\n";
     return 0;
}
```

2、 最小生成树

```
// g++ 2_2.cpp -o 2_2 && ./2_2
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;
struct DSU {
    vector<int> p, r;
    explicit DSU(int n = 0) : p(n), r(n, 0) {
        for (int i = 0; i < n; ++i) p[i] = i;  // 每个点父亲指向自己
    }
    int find(int x){ return p[x]==x?x:p[x]=find(p[x]); }
    bool unite(int a,int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(r[a]<r[b]) swap(a,b);
        p[b]=a; if(r[a]==r[b]) r[a]++;
        return true;
    }
};
// --- Kruskal ---
bool runKruskal(int n, const vector< tuple<int,int,ll> >& edges) {
    vector< tuple<ll,int,int> > es; es.reserve(edges.size());
    for (size_t i = 0; i < edges.size(); ++i) {
        int u = std::get<0>(edges[i]);
        int v = std::get<1>(edges[i]);
```

```
        ll w = std::get<2>(edges[i]);
        es.emplace_back(w, u, v);
    }
    sort(es.begin(), es.end());

    DSU dsu(n);
    ll total = 0;
    vector< tuple<int,int,ll> > mst; mst.reserve(n ? n-1 : 0);

    for (size_t i = 0; i < es.size(); ++i) {
        ll w = std::get<0>(es[i]);
        int u = std::get<1>(es[i]);
        int v = std::get<2>(es[i]);
        if (dsu.unite(u, v)) {
            mst.emplace_back(u, v, w);
            total += w;
            if ((int)mst.size() == n - 1) break;
        }
    }
    if ((int)mst.size() != n - 1) {
        cout << "图不连通，无最小生成树\n";
        return false;
    }
    cout << "Kruskal 最小生成树边集：\n";
    for (size_t i = 0; i < mst.size(); ++i) {
        int u = std::get<0>(mst[i]);
        int v = std::get<1>(mst[i]);
```

```
        ll w = std::get<2>(mst[i]);
        cout << u << " - " << v << " (" << w << ")\n";
    }
    cout << "总权值 = " << total << "\n";
    return true;
}

// --- Prim（小根堆） ---
bool runPrim(int n, const vector< vector< pair<int,ll> > >& g) {
    vector<ll> key(n, numeric_limits<ll>::max()/4);
    vector<int> parent(n, -1);
    vector<char> vis(n, 0);

    priority_queue<
        pair<ll,int>,
        vector<pair<ll,int>>,
        greater< pair<ll,int> >
    > pq;

    key[0] = 0;
    pq.push(make_pair(0LL, 0));
    while (!pq.empty()) {
        pair<ll,int> top = pq.top(); pq.pop();
        ll w = top.first;
        int u = top.second;
        (void)w; // w 未直接使用，仅为对齐原逻辑
        if (vis[u]) continue;
```

```
        vis[u] = 1;

        const vector< pair<int,ll> >& adj = g[u];
        for (size_t i = 0; i < adj.size(); ++i) {
            int v = adj[i].first;
            ll wt = adj[i].second;
            if (!vis[v] && wt < key[v]) {
                key[v] = wt; parent[v] = u;
                pq.push(make_pair(key[v], v));
            }
        }
    }

    vector< tuple<int,int,ll> > mst; mst.reserve(n ? n-1 : 0);
    ll total = 0;
    for (int v = 1; v < n; ++v) {
        if (parent[v] == -1) {
            cout << "图不连通，无最小生成树\n";
            return false;
        }
        ll w = key[v];
        mst.emplace_back(parent[v], v, w);
        total += w;
    }

    cout << "Prim 最小生成树边集：\n";
    for (size_t i = 0; i < mst.size(); ++i) {
```

```
        int u = std::get<0>(mst[i]);
        int v = std::get<1>(mst[i]);
        ll w = std::get<2>(mst[i]);
        cout << u << " - " << v << " (" << w << ")\n";
    }
    cout << "总权值 = " << total << "\n";
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    char method;
    int n, m;
    cin >> method;
    cin >> n >> m;

    vector< tuple<int,int,ll> > edges; edges.reserve(m);
    vector< vector< pair<int,ll> > > g(n);

    for (int i = 0; i < m; ++i) {
        int u, v; ll w;
        cin >> u >> v >> w;
        if (u < 0 || u >= n || v < 0 || v >= n) {
            cout << "输入不合法：顶点越界\n";
            return 0;
```

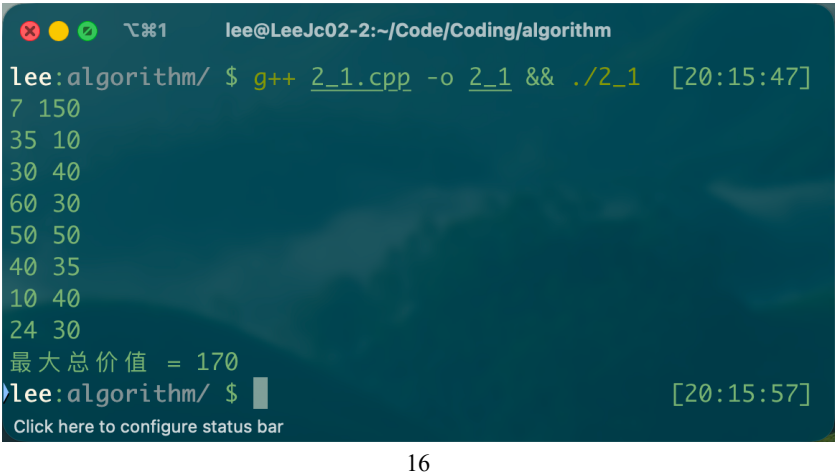
```
        }
        edges.emplace_back(u, v, w);
        g[u].push_back(make_pair(v, w));
        g[v].push_back(make_pair(u, w));
    }

    if (method == 'K') {
        runKruskal(n, edges);
    } else {
        runPrim(n, g);
    }
    return 0;
}
```

六、运行结果和算法复杂度分析

1、背包问题

（1）运行结果



（2）算法复杂度分析

递推：T(n)=2T(n/2)+n，T(1)=O(1)，得 T(n)=O(n log n)

时间复杂度：O(n log n)，**空间复杂度：**O(n)

2、最小生成树

（1）运行结果

Kruskal：



Prim：



（2）算法复杂度分析

Kruskal：

递推：T(m)=2T(m/2)+m，T(1)=O(1)，得 T(m)=O(m log m)

时间复杂度：O(m log m)，**空间复杂度：**O(n+m)

Prim：

递推：T(n)=T(n-1)+n，T(1)=O(1)，得 T(n)=O(n^2)

时间复杂度：O(n^2)，**空间复杂度：**O(n)

实验三 多段图和货郎担问题算法实现（用动态规划方法）

一、实验目的

1、掌握能用动态规划方法求解的问题应满足的条件。
2、加深对动态规划方法的理解与应用。
3、锻炼学生对程序跟踪调试能力。
4、通过本次实验的练习培养学生应用所学知识解决实际问题的能力。

二、实验内容

1、用动态规划法求解多段图问题。
2、用动态规划法求解货郎担问题。

三、实验要求

1、编写完整的动态规划算法程序，能够读取图的输入数据，并输出从源点到汇点的最短路径长度和顶点序列。

2、编写完整的动态规划算法程序，能够读取图的输入数据（4 个结点即可），可以将任意顶点设为起点，输出最佳周游路线，并给出最短距离。

四、实验过程设计（算法设计过程）

动态规划的思想实质是分治思想和解决冗余。与分治法类似的是，将原问题分解成若干个子问题，先求解子问题，再从这些子问题的解得到原问题的解。与分治法不同的是，经分解的子问题往往不是互相独立的。若用分治法来解，有些共同部分（子问题或子子问题）被重复计算了很多次。如果能够保存已解决的子问题的答案，在需要时再查找，这样就可以避免重复计算、节省时间。动态规划法用一个表来记录所有已解的子问题的答案。

动态规划方法的基本思想是，把求解的问题分成许多阶段或多个子问题，然后按顺序求解各子问题。最后一个子问题就是初始问题的解。

由于动态规划的问题有重叠子问题的特点，为了减少重复计算，对每一个子问题只解一次，将其不同阶段的不同状态保存在一个二维数组中。

1、多段图

多段图是一个有向、无环、带权图，有一个起始结点（源点）和一个终止结点（汇点）。它有 k 个阶段，每个阶段由特定的几个结点构成。每个结点的所有出边都只能指向下一个相邻的阶段，阶段之间不能越界。

设计两个主要的数据结构：

（1）COST 数组：用于记录以某个结点为起点到终点 t 的最短路径长度值。数组的下标表示结点的编号，数组的值表示以该结点为起点到终点的最短路径长度。

（2）D 数组：用于记录最短路径中出现的所有结点。下标表示结点的编号，D[i]表示结点 i 的后继结点编号。

算法流程：

（1）从最后一个结点（汇点）开始，从后向前依次计算每个结点的 COST 值和 D 值。

（2）计算每个结点的 COST 值时，找到该结点所有的出边，分别计算每条边到汇点的路径长度，并选择最小值作为当前结点的 COST 值。

（3）同时更新 D 数组，记录每个结点的后继结点。

（4）重复上述步骤，直到计算完第一个结点（源点），即可根据 D 数组得到最短路径。

2、货郎担

货郎担问题，也称为旅行商问题（TSP 问题），是指有一个货郎需要访问 n 个城市，每个城市恰好访问一次，最后返回起点城市，目标是找到一条最短路径，使得总路程最短。

算法流程：

（1）设 g(i, S)表示从城市 i 出发，访问集合 S 中的城市后再返回 i 的最短路径长度。递推公式为：$g(i, S) = \min\{cij + g(j, S-\{j\})\}$，其中 cij 是城市 i 到 j 的距离。

（2）对于任意城市 k，从城市 i 出发不经过任何城市直接回到 i 的距离是 0，因此 $g(i, \{i\}) = 0$。

（3）通过递归或迭代的方式计算所有可能的子集，直到计算出从任意城市出发访问所有城市并返回起点的最短路径。

五、源代码

1、多段图

```
// g++ 3_1.cpp -o 3_1 && ./3_1
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int k, n, m, s, t;
    cin >> k >> n >> m >> s >> t;
    vector<int> stage(n);
    vector<vector<pair<int,ll>>> g(n);

    for (int i = 0; i < n; ++i) cin >> stage[i];
    for (int i = 0; i < m; ++i) {
```

```
    int u,v; ll w; cin >> u >> v >> w;
    if (u<0||u>=n||v<0||v>=n) continue;
    // 仅保留指向下一阶段的边
    if (stage[v] == stage[u] + 1) g[u].push_back({v, w});
}

const ll INF = numeric_limits<ll>::max()/4;
vector<vector<int>> byStage(k);
for (int u = 0; u < n; ++u) {
    if (stage[u]>=0 && stage[u] < k) byStage[stage[u]].push_back(u);
}

vector<ll> cost(n, INF);
vector<int> nxt(n, -1);
cost[t] = 0;
for (int st = k-2; st >= 0; --st) {
    for (int u : byStage[st]) {
        for (const auto& e : g[u]) {
            int v = e.first;
            long long w = e.second;
            if (cost[v] + w < cost[u]) {
                cost[u] = cost[v] + w;
                nxt[u] = v;
            }
        }
    }
}
```

```
    cout << "最短路径长度 = " << cost[s] << "\n 路径: ";
    vector<int> path; path.push_back(s);

    for (int u = s; u != t; u = nxt[u]) {
        if (u == -1) break;
        path.push_back(nxt[u]);
    }

    for (int i = 0; i < (int)path.size(); ++i) {
        cout << path[i] << (i+1==(int)path.size()?'\n':' ');
    }
     return 0;
}
```

2、货郎担

```
// g++ 3_2.cpp -o 3_2 && ./3_2
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, s;
```

```
    cin >> n >> s;
    const ll INF = numeric_limits<ll>::max()/4;
    vector<vector<ll>> w(n, vector<ll>(n));
    for(int i=0;i<n;++i) for(int j=0;j<n;++j){
        ll x; cin >> x; w[i][j] = (x < 0 ? INF : x);
    }

    int FULL = (1<<n); // 2^n
    vector<vector<ll>> dp(FULL, vector<ll>(n, INF));
    vector<vector<int>> parent(FULL, vector<int>(n, -1));
    dp[1<<s][s] = 0;

    for (int mask = 0; mask < FULL; ++mask) {
        if (!(mask & (1<<s))) continue; // 必须包含起点
        for (int u = 0; u < n; ++u) if (dp[mask][u] < INF) {
            for (int v = 0; v < n; ++v) if (!(mask & (1<<v)) && w[u][v]
< INF) {
                int nmask = mask | (1<<v);
                ll cand = dp[mask][u] + w[u][v];
                if (cand < dp[nmask][v]) {
                    dp[nmask][v] = cand;
                    parent[nmask][v] = u;
                }
            }
        }
    }
```

```
    ll best = INF; int last = -1; int ALL = FULL - 1;

    for (int j = 0; j < n; ++j) if (j != s && dp[ALL][j] < INF && w[j][s]
< INF) {
        ll cand = dp[ALL][j] + w[j][s];
        if (cand < best) { best = cand; last = j; }
    }
    // 路径还原
    vector<int> route; route.push_back(s);
    int mask = ALL, v = last;
    vector<int> rev;
    while (v != s) {
        rev.push_back(v);
        int u = parent[mask][v];
        mask ^= (1<<v);
        v = u;
    }

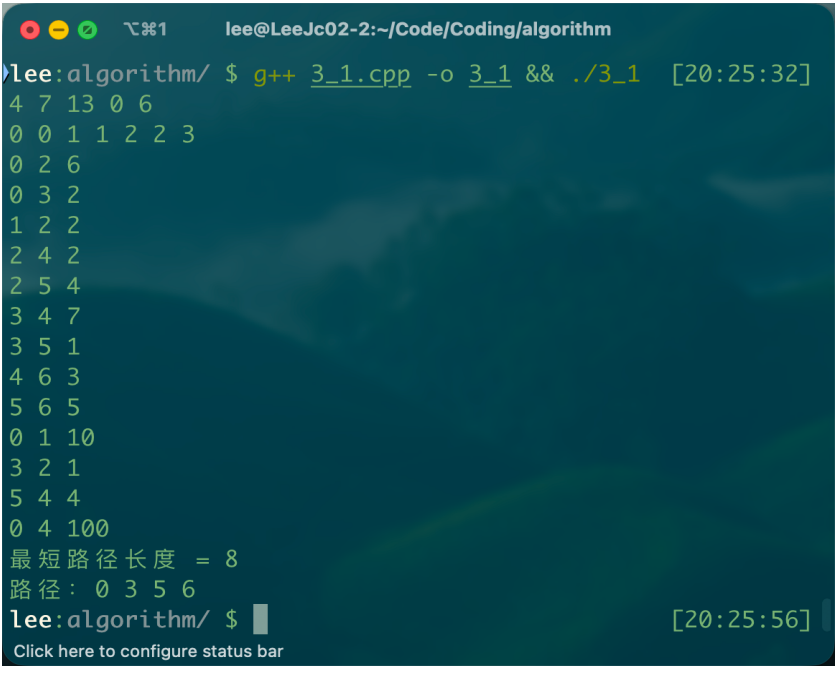
    reverse(rev.begin(), rev.end());
    for (int x : rev) route.push_back(x);
    route.push_back(s);

    cout << "最短回路长度 = " << best << "\n 回路: ";
    for (int i = 0; i < (int)route.size(); ++i)
        cout << route[i] << (i+1==(int)route.size()?'\n':' ');
    return 0;
}
```

六、运行结果和算法复杂度分析

1、多段图

（1）运行结果



（2）算法复杂度分析

递推：T(m)=T(m-1)+O(1)，T(1)=O(1)，得 T(m)=O(m)

时间复杂度：O(V+E)，**空间复杂度：**O(V)

2、货郎担

（1）运行结果



（2）算法复杂度分析

递推：T(n)=2T(n-1)+O(n•2^{n-1})，T(1)=O(1)，得T(n)=O(n^2•2^n)

时间复杂度：O(n^2•2^n)，**空间复杂度：**O(n•2^n)

实验四 皇后与子集和数问题算法实现（用回溯法）

一、实验目的

1、掌握能用回溯法求解的问题应满足的条件。
2、加深对回溯法算法设计方法的理解与应用。
3、锻炼学生对程序跟踪调试能力。
4、通过本次实验的练习培养学生应用所学知识解决实际问题的能力。

二、实验内容

1、使用回溯法找出 n 皇后问题的所有解。
2、使用回溯法求解子集和数问题。

三、实验要求

1、由 N2 个方块排成 N 行 N 列的正方形，称为 N 元棋盘，在 N 元棋盘上放置 N 个皇后，如果某两个皇后位于 N 元棋盘的同一行或同一列或同一斜线（斜率为±1）上，则称它们在互相攻击，使用回溯法找出使 N 个皇后互不攻击的所有布局。

2、输入任意整数构成的集合，并确定和数 M，使用回溯法从该集合中找出使和为 M 的所有子集。

四、实验过程设计（算法设计过程）

回溯法是一种通过逐步构建解决方案并回溯到之前的步骤来尝试其他可能性，直到找到所有可能的解决方案或确定无解的方法。

1、皇后问题

在 n×n 的棋盘上，需要放置 n 个皇后，使得任意两个皇后都不能处于同一行、同一列或同一斜线上。

算法流程：

（1）确定解空间的起点，通常是在第一行第一列放置一个皇后，并初始化其他变量。

（2）从第二行开始，在每一列依次尝试放置皇后，并进行递归搜索。在搜索过程中，对于每一步，做出一个选择并进行检查，然后继续向下一步递归搜索。

（3）在每次尝试放置皇后时，检查该位置是否与已经放置的皇后冲突。如果冲突，则不满足约束条件。

（4）如果当前位置无法放置皇后（即无法满足约束条件），则回溯到上一步，尝试下一个位置。回溯是回溯法的核心，它允许算法在发现当前选择不可行时，能够撤销上一步的选择，并尝试其他可能性。

（5）当成功放置了 n 个皇后，或者已经尝试了所有可能的位置时，终止搜索。如果成功放置了 n 个皇后，则找到了一个满足条件的解；如果尝试了所有可能的位置仍未找到解，则确定无解。

2、子集和数问题

在子集和数问题中，给定一个正整数集合 $X=x_1,x_2,\ldots,x_n$ 和一个正整数 y，目标是找出 X 的一个子集 Y，使得 Y 中元素的和等于 y。

算法流程：

（1）输入两个整数 n 和 y，分别表示集合 X 的长度和目标整数 y。然后输入 n 个整数，表示集合 X 中的元素；设置一个数组 t[]用于存储当前的子集和。

（2）设置一个标志变量 flag，flag=0 表示尚未找到解，flag=1 表示找到解。

（3）定义一个回溯函数 backtrack(int index, int sum, int n)，其中 index 表示当前处理的元素索引，sum 表示当前子集的和，n 表示集合的长度。

（4）如果 index > n，则检查 sum 是否等于 y，如果等于则输出解，并设置 flag 为 1，否则继续回溯。

（5） 在 backtrack 函数中，首先尝试将当前元素加入子集并递归调用 backtrack(index + 1, sum +t[index], n)，然后撤销选择并继续尝试其他可能性。

五、源代码

1、皇后问题

```
// g++ 4_1.cpp -o 4_1 && ./4_1
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
using ull = unsigned long long;

int n;
vector<int> colAt; // 第 row 行皇后所在列
long long solutions = 0;
int cnt = 0;

void dfs(int row, ull cols, ull diag1, ull diag2){
    if (row == n) {
        solutions++;
        // 输出一个解
        cout << "第" << solutions << "个解是：" << endl;
        for (int i = 0; i < n; ++i) {
            string line(n, '.');
            line[colAt[i]] = 'Q';
            cout << line << "\n";
        }
        cout << "\n";
```

```
        return;
    }
    const ull lowN = (n == 64 ? ~0ULL : ((1ULL << n) - 1ULL));
    ull avail = (~(cols | diag1 | diag2)) & lowN;
    while (avail) {
        ull p = avail & -avail; // 取最低可用位
        int c = __builtin_ctzll(p);
        colAt[row] = c;
        dfs(row+1, cols|p, (diag1|p)<<1, (diag2|p)>>1);
        avail ^= p;
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    colAt.assign(n, -1);
    dfs(0, 0, 0, 0);
    cout << "解的数量 = " << solutions << "\n";
    return 0;
}
```

2、子集和数问题

```
// g++ 4_2.cpp -o 4_2 && ./4_2
#include <iostream>
```

```
#include <vector>
#include <algorithm>
using namespace std;
using ll = long long;

int n; ll M;
vector<ll> a;
vector<ll> posSuf, negSuf; // 从 i 到末尾，正数和 & 负数和
vector<ll> cur;
ll solutions = 0;

void dfs(int i, ll sum){
    if (i == n) {
        if (sum == M) {
            solutions++;
            cout << "{";
            for (int k = 0; k < (int)cur.size(); ++k) {
                cout << cur[k] << (k+1==(int)cur.size()?"":", ");
            }
            cout << "}\n";
        }
        return;
    }
    ll minReach = sum + negSuf[i];
    ll maxReach = sum + posSuf[i];
    if (M < minReach || M > maxReach) return;
```

```
    cur.push_back(a[i]);
    dfs(i+1, sum + a[i]);
    cur.pop_back();

    dfs(i+1, sum);
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> M;
    a.resize(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    sort(a.begin(), a.end());
    posSuf.assign(n+1, 0);
    negSuf.assign(n+1, 0);
    for (int i = n-1; i >= 0; --i) {
        posSuf[i] = posSuf[i+1] + (a[i] > 0 ? a[i] : 0);
        negSuf[i] = negSuf[i+1] + (a[i] < 0 ? a[i] : 0);
    }
    dfs(0, 0);
    if (solutions == 0) cout << "无解\n";
    else cout << "解的数量 = " << solutions << "\n";
    return 0;
}
```

六、运行结果和算法复杂度分析

1、皇后问题

（1）运行结果





（2）算法复杂度分析

递推：T(n)=n•T(n-1)+0(1)，T(1)=0(1)，得T(n)=0(n!)

时间复杂度：0(n!)，**空间复杂度：**0(n)

2、子集和数问题

（1）运行结果



（2）算法复杂度分析

递推：T(n)=2T(n-1)+0(1)，T(1)=0(1)，得 T(n)=0(2^n)

时间复杂度：0(2^n)，**空间复杂度：**0(n)